

Module 5 Week B — Stretch: Hyperparameter Tuning & Nested Cross-Validation

Honors Track. This is an Honors Track stretch assignment. Stretch assignments are for learners who have completed all core assignments, are On Track or Advanced, and are attending consistently. If you are behind on core work, focus there first.

Prerequisite: Lab 5B base + at least Lab 5B Challenge Tier 1 (Threshold Tuning). This stretch builds on the systematic evaluation skills from Tier 1. If you jumped straight from the base lab to this stretch, go back and complete Tier 1 first — the threshold sweep gives you the foundation for interpreting grid search results.

Due: Thursday end of day. Resubmissions are accepted through Saturday of the assignment week.

The Challenge

This stretch has two parts. Part 1 implements systematic hyperparameter tuning with `GridSearchCV`. Part 2 reveals a problem with Part 1 that most ML practitioners never notice: the score `GridSearchCV` reports is optimistically biased because it uses the same data for selecting hyperparameters and estimating performance. You will implement nested cross-validation to get an honest score and discover how large the bias actually is — and why it depends on the model family.

Part 1 — GridSearchCV

Implement systematic hyperparameter tuning for the random forest model using `GridSearchCV`. Define a parameter grid, run an exhaustive search with stratified cross-validation, visualize the results, and analyze what the grid search reveals about model complexity for this dataset.

Manual hyperparameter selection (trying a few values and picking the best) does not scale. `GridSearchCV` systematically evaluates every combination in a parameter grid, using cross-validation to estimate generalization performance at each point.

Parameter grid (Random Forest)

Hyperparameter	Values
<code>n_estimators</code>	50, 100, 200
<code>max_depth</code>	3, 5, 10, 20, None
<code>min_samples_split</code>	2, 5, 10

Constraints

- Use `GridSearchCV` with 5-fold stratified cross-validation and `scoring='f1'`
- Produce a heatmap showing F1 scores across the `max_depth` x `n_estimators` grid (aggregate across `min_samples_split` values or fix `min_samples_split` at the best value — your choice, but justify it)
- Report `best_params` and `best_score` from the fitted grid search
- The total grid has $3 \times 5 \times 3 = 45$ combinations $\times$ 5 folds = 225 fits — this should run in a reasonable time on the telecom dataset

Part 1 analysis (1 paragraph)

- Which hyperparameters have the largest impact on F1 score?
- Is there a clear “sweet spot,” or does performance plateau across a range of values?
- Does the grid search suggest the model is more at risk of underfitting or overfitting for this dataset?
- Would you expand the grid in any direction based on these results? If so, where and why?

Part 2 — Nested Cross-Validation

`GridSearchCV.best_score_` is the average F1 across the inner CV folds — but the hyperparameters were *selected* to maximize that exact score. This is the same problem as evaluating a model on its training data: the estimate is optimistically biased because the same data informed both the selection and the evaluation.

The fix is **nested cross-validation**: an outer CV loop evaluates the entire tuning procedure as a black box, and an inner CV loop (the `GridSearchCV`) selects hyperparameters on each outer training fold. The outer scores are honest — they evaluate models whose hyperparameters were chosen without ever seeing the outer test fold.

What to implement

- Outer loop:** 5-fold `StratifiedKFold` (use a different `random_state` than the inner `GridSearchCV` so the folds are not identical)
- Inner loop:** `GridSearchCV` with the same parameter grid and settings from Part 1
- For each outer fold:
 - Fit `GridSearchCV` on the outer training set (this runs the full inner grid search)
 - Score the best model from the inner search on the outer test fold
 - Record: the outer test F1, and the inner `best_score` for that fold
- Run nested CV on two model families:**
 - `RandomForestClassifier` (with `class_weight='balanced'`) — the same grid from Part 1
 - `DecisionTreeClassifier` (with `class_weight='balanced'`) — use a grid of `max_depth: [3, 5, 10, 20, None]` and `min_samples_split: [2, 5, 10]` (15 combos $\times$ 5 inner folds $\times$ 5 outer folds = 375 fits — fast on this dataset)

What to compare

For each model family, report:

Metric	Random Forest	Decision Tree
Inner <code>best_score</code> (mean across 5 outer folds)	?	?
Outer nested CV score (mean across 5 outer folds)	?	?
Gap (inner - outer)	?	?

The gap is the selection bias — how much the inner CV overestimates performance because the hyperparameters were tuned on the same data.

Part 2 analysis (1-2 paragraphs)

- Which model family shows a larger gap between inner `best_score` and outer nested score? Why does this make sense given what you know about how decision trees and random forests handle variance?
- Is the `GridSearchCV.best_score_` from Part 1 trustworthy for the random forest? What about for the decision tree? Cite the specific gap values.
- Connect this to the Module 5 Week A takeaway about held-out test sets: the same principle applies here — data that informed a decision (hyperparameter selection) cannot also honestly evaluate that decision. How is nested CV the hyperparameter-tuning equivalent of the held-out test set from the Week A integration?

Deliverables

Your deliverable is a standalone repository containing:

Artifact	Description
Python script or notebook	Parts 1 and 2, clearly separated
Heatmap visualization	F1 scores across the <code>max_depth</code> $\times$ <code>n_estimators</code> grid (Part 1)
Nested CV comparison table	Inner vs. outer scores for both model families (Part 2)
Analysis	Part 1 paragraph + Part 2 paragraphs (see analysis prompts above)

Tips

- If performance plateaus at `max_depth=None`, the model may benefit from deeper investigation into regularization parameters (`min_samples_leaf`, `max_features`) rather than just tree depth.
- The heatmap should use a diverging or sequential colormap that makes small F1 differences visible. Consider annotating cells with the actual F1 values.
- `GridSearchCV` stores all results in `cv_results` — use it to build your heatmap rather than re-running models.
- For nested CV, the outer loop runs 5 complete grid searches. With the random forest grid (45 combos $\times$ 5 inner folds $\times$ 5 outer folds = 1,125 fits) plus the decision tree grid (375 fits), the total is ~1,500 fits. On the telecom dataset (~1,500 rows), expect this to take a few minutes, not hours.
- The decision tree’s gap will likely be noticeably larger than the random forest’s. This is not a bug — it is the point. Decision trees have high variance, so their best hyperparameters are more sensitive to the specific training data. Random forests reduce variance through bagging, which also reduces selection bias.
- Think about what the gap would look like on a much larger dataset (100k rows) vs. this one. Selection bias shrinks with more data — why?

Submission

- Create a standalone repository for this stretch assignment (not a GitHub Classroom repo — create your own)
- Push your script, heatmap, nested CV table, and analysis to the repository
- Paste your repository URL into TalentLMS → Module 5 Week B → Stretch 5B-S1

